



Assessment Report
on
“AI/ML Prediction-Based”
submitted as partial fulfillment for the award of
BACHELOR OF TECHNOLOGY
DEGREE

SESSION 2025-26

in
CSE(AIML)

By

Name: Shreya

Roll Number: 202401100400179

Section: C

Under the supervision of
“Shivansh Prasad”

KIET Group of Institutions, Ghaziabad

May, 2026

Problem 1: Titanic Survival Prediction

1. Introduction

The Titanic Survival Prediction project aims to predict whether a passenger survived the Titanic disaster using machine learning classification techniques. The dataset contains passenger information such as age, gender, ticket fare, passenger class, and family details. Data preprocessing and Naive Bayes classification are used to build the prediction model.

2. Problem Statement

To develop a classification model that predicts passenger survival in the Titanic dataset using Naive Bayes classification and data preprocessing techniques.

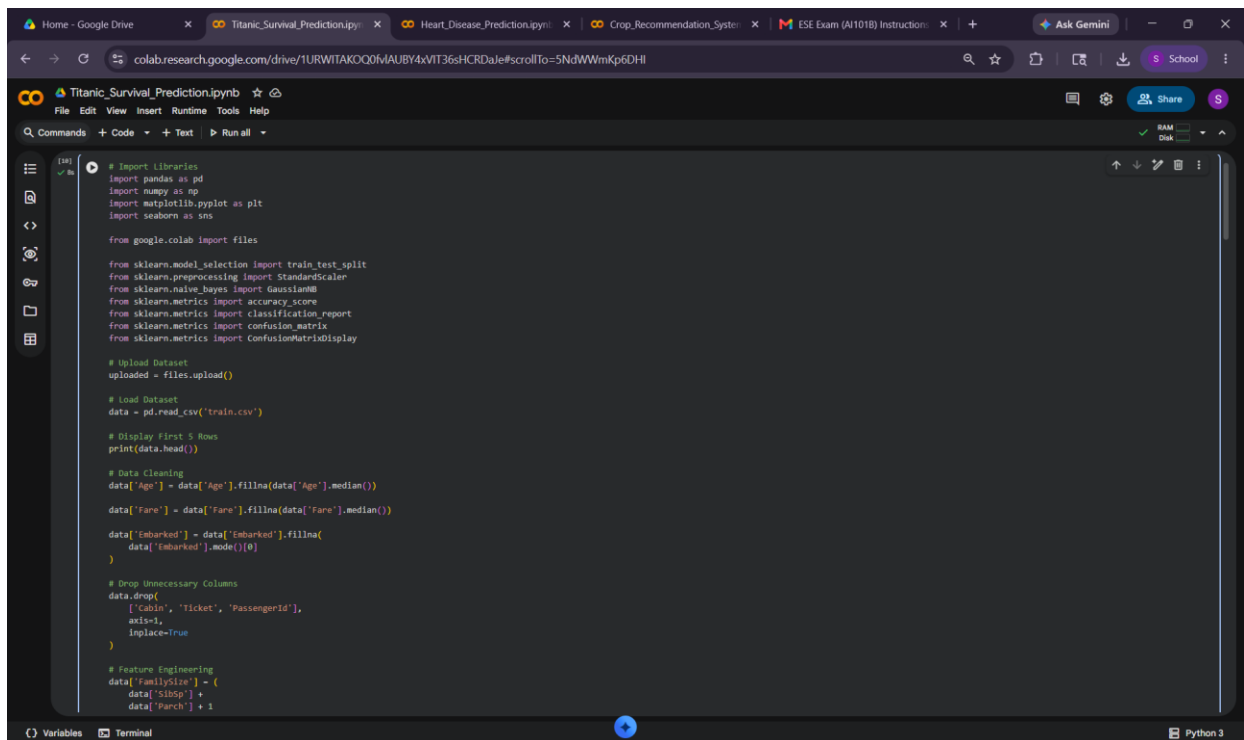
3. Objectives

- Clean and preprocess the dataset
 - Handle missing values
 - Train a Naive Bayes classification model
 - Predict passenger survival
 - Visualize the results using graphs and confusion matrix
-

4. Methodology

- **Data Collection:** The Titanic dataset was downloaded from Kaggle and uploaded into Google Colab.
 - **Data Preprocessing:**
 - Missing values in Age and Fare columns were filled using median values
 - Embarked missing values were filled using mode
 - Unnecessary columns were removed
 - Categorical data was converted into numerical form
 - New features such as FamilySize and IsAlone were created
 - **Model Building:**
 - Dataset was split into training and testing sets
 - Feature scaling was performed using StandardScaler
 - Gaussian Naive Bayes classifier was trained
 - **Model Evaluation:**
 - Accuracy score was calculated
 - Classification report was generated
 - Confusion matrix heatmap was visualized
-

5. Code



The screenshot shows a Google Colab notebook titled "Titanic_Survival_Prediction.ipynb". The code in the first cell includes the following steps:

```
[In] 1 # Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

# Upload Dataset
uploaded = files.upload()

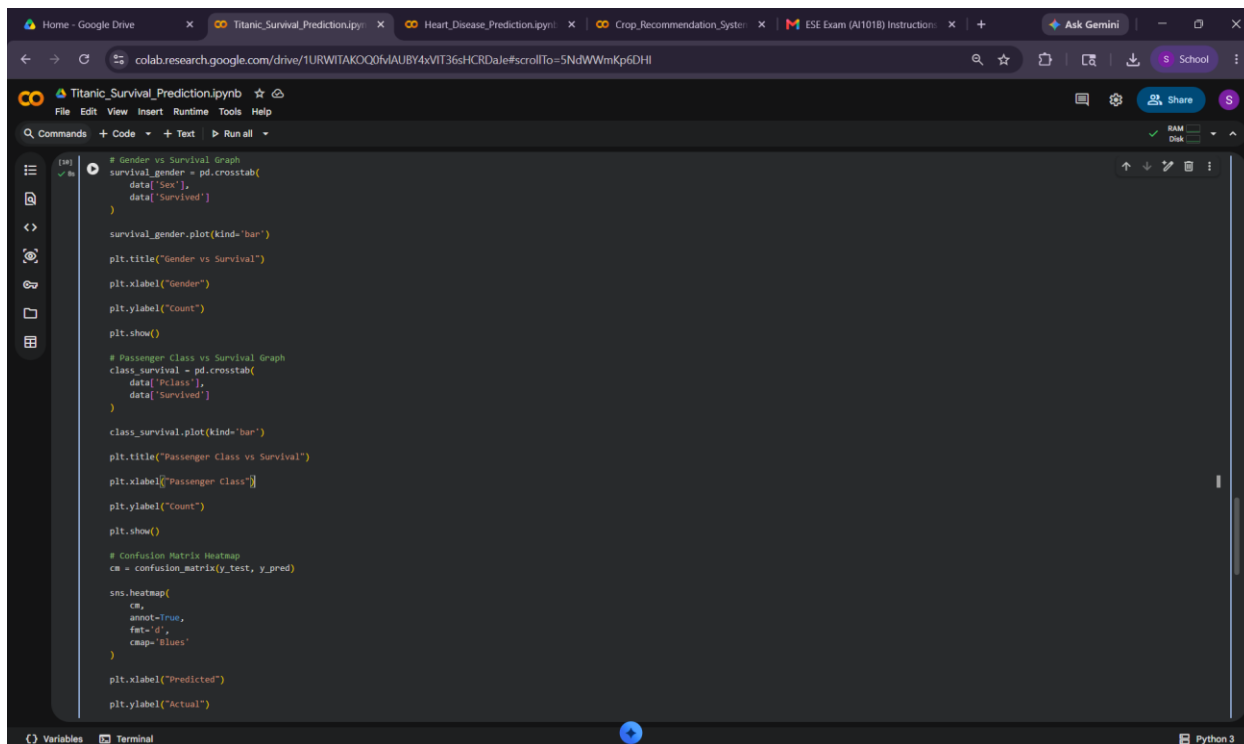
# Load Dataset
data = pd.read_csv('train.csv')

# Display First 5 Rows
print(data.head())

# Data Cleaning
data['Age'] = data['Age'].fillna(data['Age'].median())
data['Fare'] = data['Fare'].fillna(data['Fare'].median())
data['Embarked'] = data['Embarked'].fillna(
    data['Embarked'].mode()[0]
)

# Drop Unnecessary Columns
data.drop(
    ['Cabin', 'ticket', 'PassengerId'],
    axis=1,
    inplace=True
)

# Feature Engineering
data['FamilySize'] = (
    data['SibSp'] +
    data['Parch'] + 1
```



The screenshot shows the continuation of the Google Colab notebook. The code in the second cell includes the following steps:

```
[In] 2 # Gender vs Survival Graph
survival_gender = pd.crosstab(
    data['Sex'],
    data['Survived']
)

survival_gender.plot(kind='bar')

plt.title("Gender vs Survival")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()

# Passenger Class vs Survival Graph
class_survival = pd.crosstab(
    data['Pclass'],
    data['Survived']
)

class_survival.plot(kind='bar')

plt.title("Passenger Class vs Survival")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
```

6. Model Implementation

Gaussian Naive Bayes algorithm was implemented for classification. The model was trained on the processed Titanic dataset and used to predict passenger survival on test data.

7. Evaluation Metrics

Evaluation Metrics

The following metrics were used:

- Accuracy Score
 - Precision
 - Recall
 - F1 Score
 - Confusion Matrix
-

8. Results and Analysis

The model achieved satisfactory accuracy in predicting passenger survival. Feature engineering and preprocessing improved prediction performance. Graphs and confusion matrix heatmap helped visualize the classification results effectively.

9. Conclusion

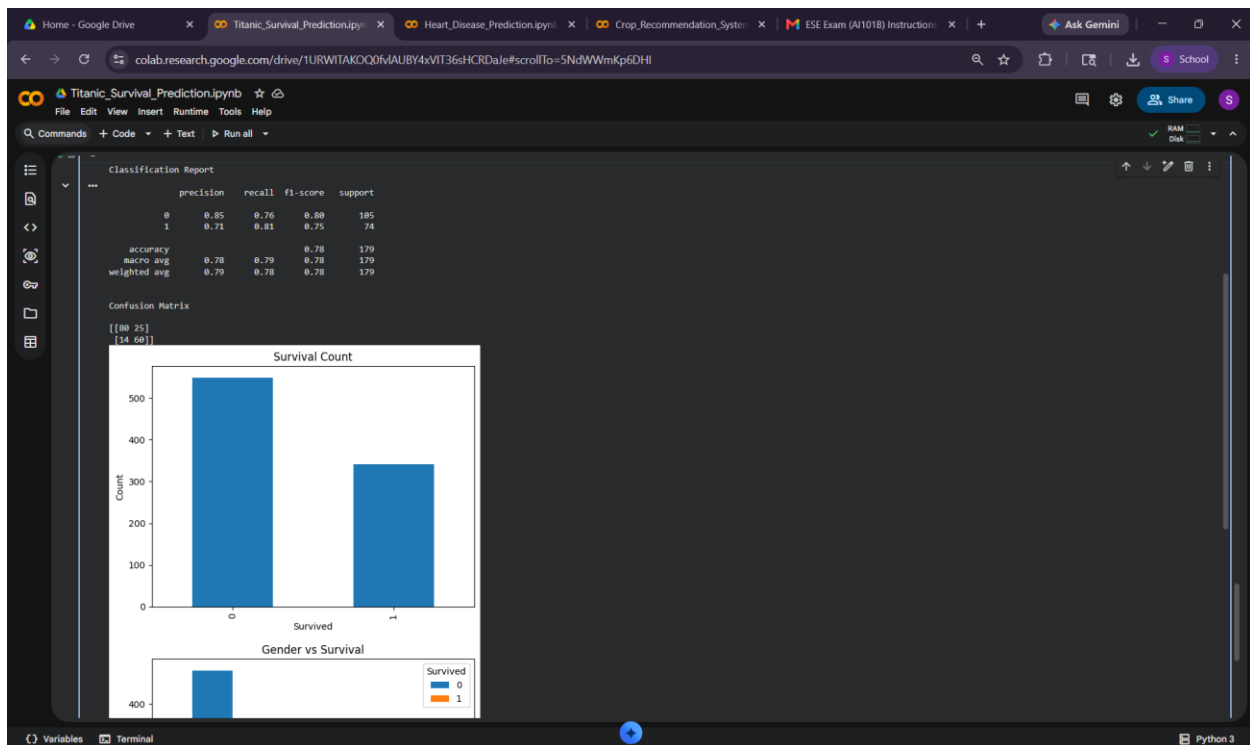
The Naive Bayes classification model successfully predicted passenger survival using the Titanic dataset. Data preprocessing and feature engineering improved model performance and helped achieve satisfactory accuracy.

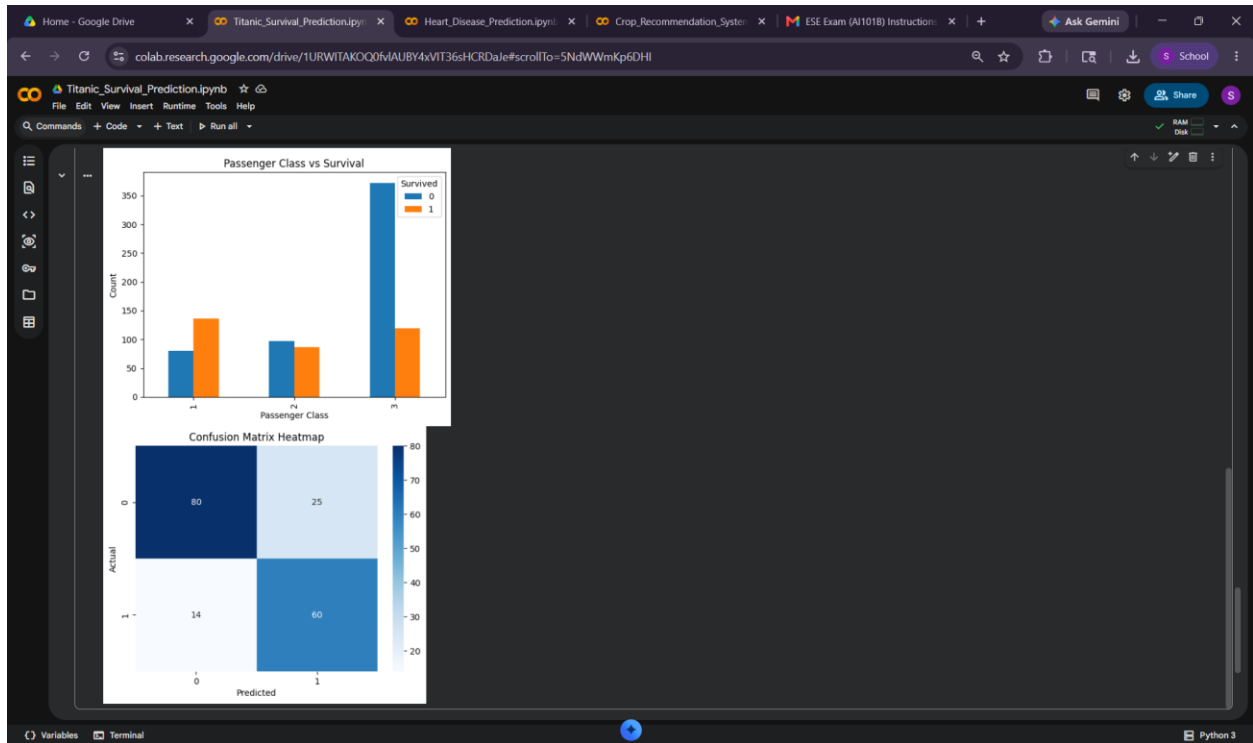
10. References

- Titanic Dataset from Kaggle

- scikit-learn documentation
- pandas documentation
- matplotlib and seaborn libraries

11. Output/Results





Problem 2: Heart Disease Prediction

1. Introduction

The Heart Disease Prediction project predicts the likelihood of heart disease based on medical parameters such as age, cholesterol level, chest pain type, blood pressure, and heart rate. Machine learning classification techniques were used to analyze the medical dataset and predict heart disease accurately.

2. Problem Statement

To develop a machine learning model that predicts heart disease using patient medical data and classification algorithms.

3. Objectives

- Preprocess medical dataset
 - Handle categorical and numerical data
 - Train a classification model
 - Predict heart disease
 - Evaluate model performance
 - Visualize predictions and relationships using graphs
-

4. Methodology

- **Data Collection:**

The Heart Disease dataset was downloaded from Kaggle and uploaded into Google Colab.

- **Data Preprocessing:**

- Missing values were removed from the dataset
- Categorical values such as gender, chest pain type, slope, and thal were encoded into numerical values
- Feature scaling was applied using StandardScaler
- Input features and target variables were separated

- **Model Building:**

- Dataset was divided into training and testing sets
- Random Forest classifier was selected for classification
- Model was trained using training data

- **Model Implementation:**

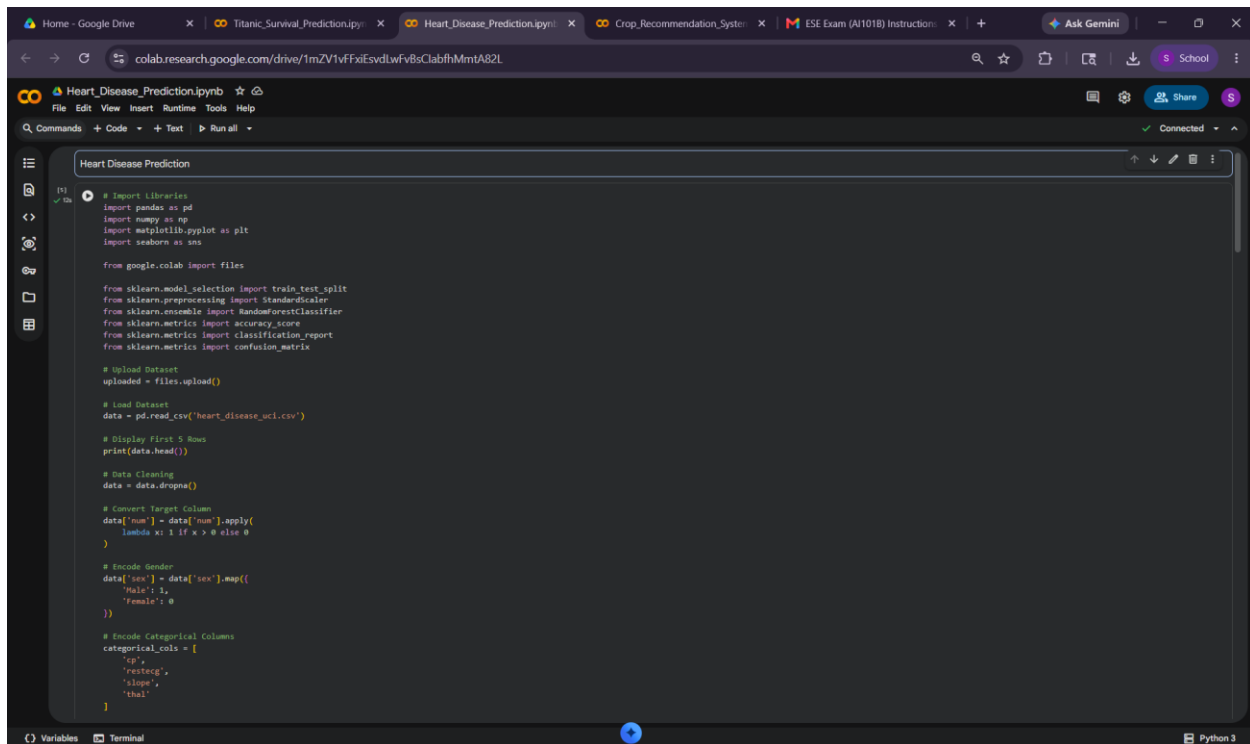
Random Forest classifier was implemented because it provides high accuracy and handles medical datasets efficiently. Multiple decision trees were trained to improve prediction accuracy.

5. Evaluation Metrics

The following evaluation metrics were used:

- Accuracy Score
- Precision
- Recall
- F1 Score
- Confusion Matrix

6. Code



```
Heart Disease Prediction

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

# Upload Dataset
uploaded = files.upload()

# Load Dataset
data = pd.read_csv('heart_disease_uci.csv')

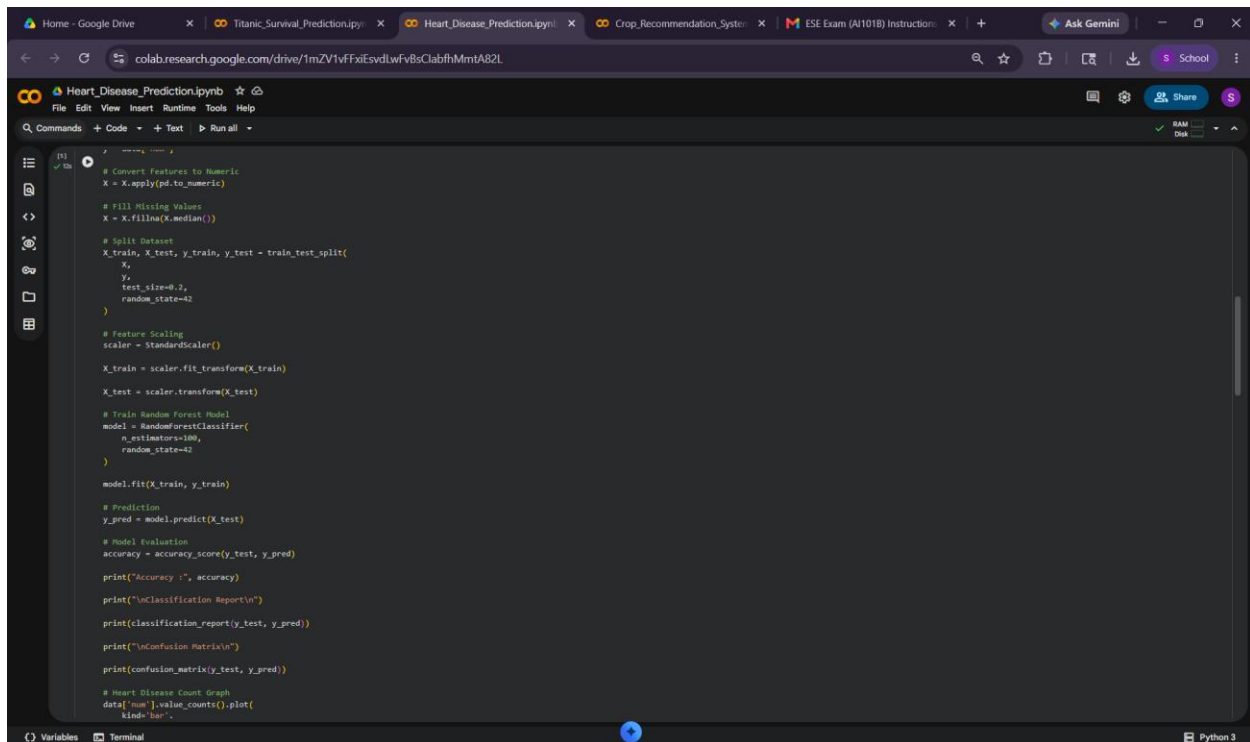
# Display First 5 Rows
print(data.head())

# Data Cleaning
data = data.dropna()

# Convert Target Column
data['num'] = data['num'].apply(
    lambda x: 1 if x > 0 else 0
)

# Encode Gender
data['sex'] = data['sex'].map({
    'Male': 1,
    'female': 0
})

# Encode Categorical Columns
categorical_cols = [
    'cp',
    'restecg',
    'slope',
    'thal'
]
```



```
# Convert Features to Numeric
X = X.apply(pd.to_numeric)

# Fill Missing Values
X = X.fillna(X.median())

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Random Forest Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Model Evaluation
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy :", accuracy)

print("\nClassification Report\n")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix\n")
print(confusion_matrix(y_test, y_pred))

# Heart Disease Count Graph
data['num'].value_counts().plot(
    kind='bar'
```

7. Results and Analysis

The Random Forest classifier achieved high accuracy in detecting heart disease. Graphs such as heart disease count, chest pain analysis, and confusion matrix heatmap helped analyze prediction performance and dataset patterns.

8. Conclusion

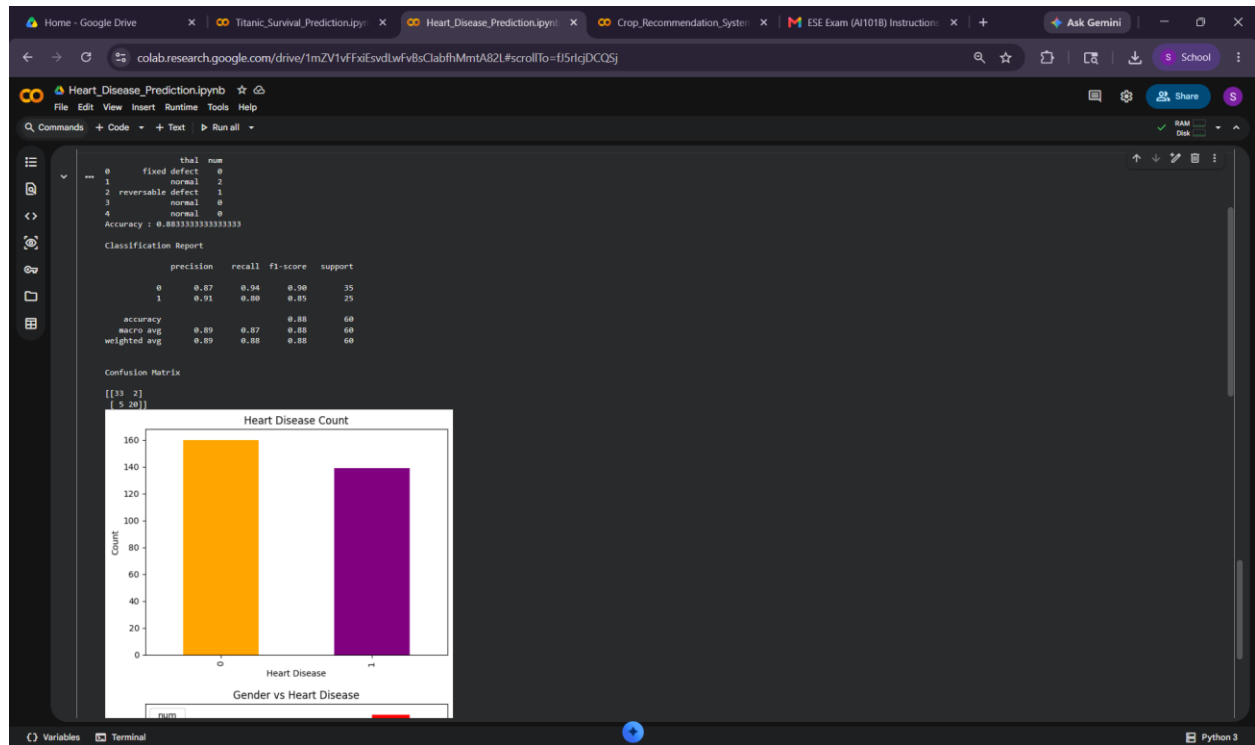
The Heart Disease Prediction system successfully predicted heart disease using machine learning classification techniques. The project demonstrates the practical application of machine learning in healthcare and disease diagnosis.

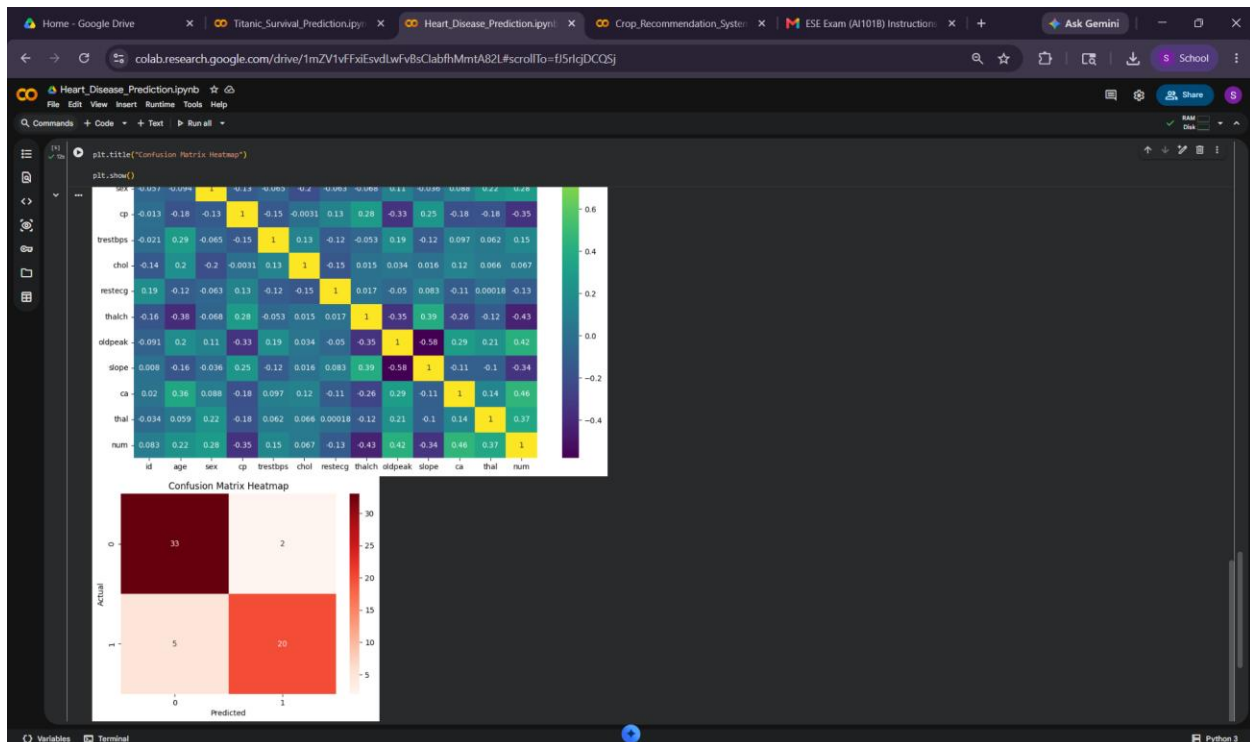
9. References

- Heart Disease Dataset from Kaggle
- scikit-learn documentation
- pandas documentation

- seaborn and matplotlib libraries

10. Output/Results





Problem 3: Crop Recommendation System

1. Introduction

The Crop Recommendation System predicts the most suitable crop based on soil nutrients and environmental conditions such as nitrogen, phosphorus, potassium, humidity, rainfall, and temperature. Machine learning classification techniques were used to recommend crops accurately.

2. Problem Statement

To design a machine learning recommendation system that predicts suitable crops based on soil and weather conditions.

3. Objectives

- Analyse agricultural dataset

- Preprocess soil and weather data
 - Train a machine learning classification model
 - Recommend suitable crops
 - Visualize feature relationships and predictions
 - Evaluate model performance
-

4. Methodology

- **Data Collection:**

The Crop Recommendation dataset was downloaded from Kaggle and uploaded into Google Colab.

- **Data Preprocessing:**

- Dataset was checked for missing values
- Features and target variables were separated
- Feature scaling was performed using StandardScaler
- Data was converted into machine learning compatible format

- **Model Building:**

- Dataset was split into training and testing sets
- Random Forest classifier was used for crop prediction
- Model was trained using processed agricultural data

- **Model Implementation:**

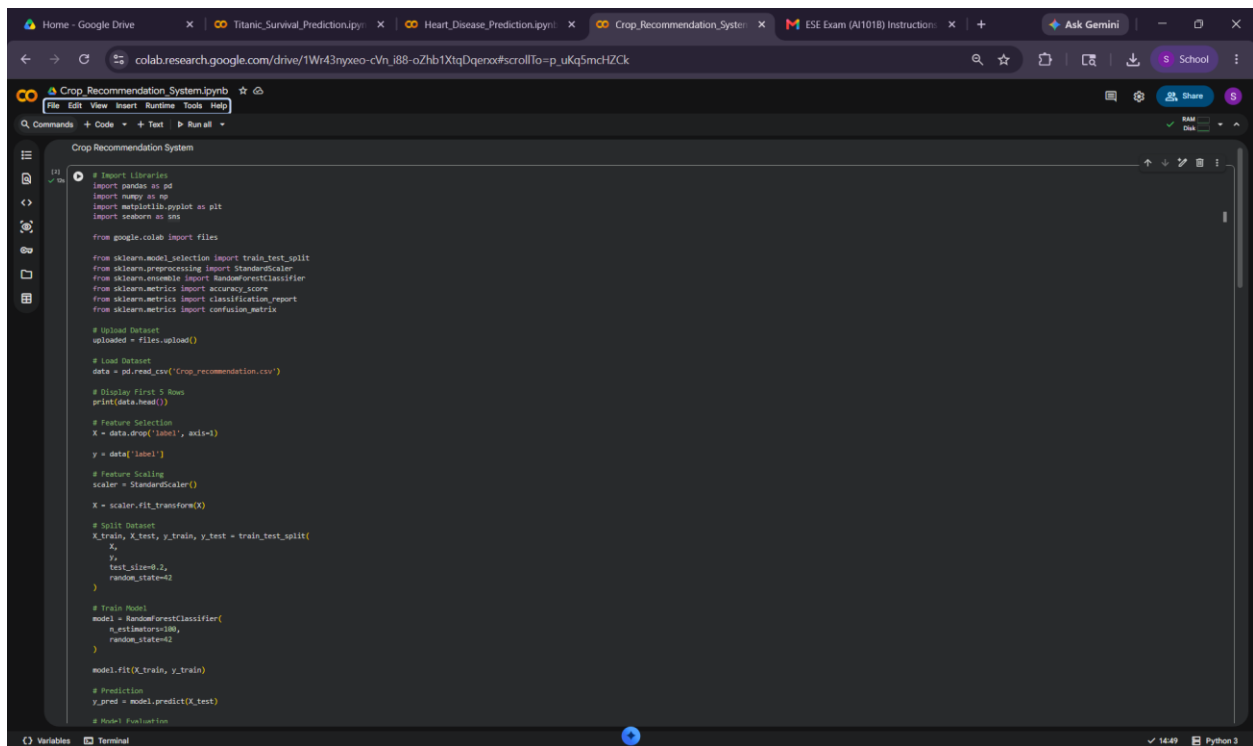
Random Forest classification algorithm was implemented because it provides high prediction accuracy and handles agricultural datasets effectively. The trained model was used to recommend suitable crops based on environmental conditions.

5. Evaluation Metrics

The following metrics were used:

- Accuracy Score
- Precision
- Recall
- F1 Score
- Confusion Matrix

6. Code



```
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

# Upload Dataset
uploaded = files.upload()

# Load Dataset
data = pd.read_csv('Crop_recommendation.csv')

# Display First 5 Rows
print(data.head())

# Feature Selection
X = data.drop('label', axis=1)
y = data['label']

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

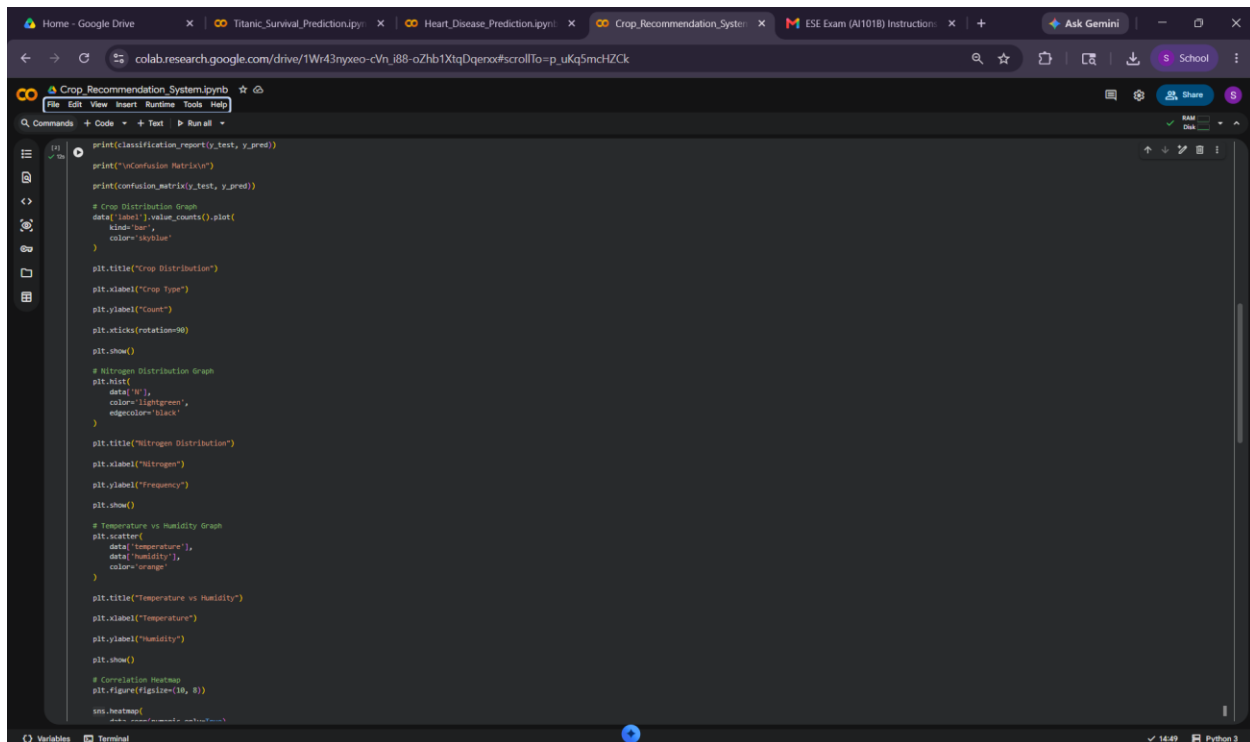
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Train Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Model Evaluation
```



```
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))

# Crop Distribution Graph
data['crop'].value_counts().plot(
    kind='bar',
    color='skyblue'
)

plt.title("Crop Distribution")
plt.xlabel("Crop Type")
plt.ylabel("Count")
plt.xticks(rotation=90)
plt.show()

# Nitrogen Distribution Graph
plt.hist(
    data['N'],
    color='lightgreen',
    edgecolor='black'
)

plt.title("Nitrogen Distribution")
plt.xlabel("Nitrogen")
plt.ylabel("Frequency")
plt.show()

# Temperature vs Humidity Graph
plt.scatter(
    data['temperature'],
    data['humidity'],
    color='orange'
)

plt.title("Temperature vs Humidity")
plt.xlabel("Temperature")
plt.ylabel("Humidity")
plt.show()

# Correlation Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(
    data[['temperature', 'humidity', 'nitrogen', 'ph', 'pest_diseases']])
```

7. Results and Analysis

The Crop Recommendation model achieved very high prediction accuracy. Graphs such as crop distribution, nitrogen distribution, temperature vs humidity graph, and confusion matrix heatmap helped visualize dataset patterns and model predictions effectively.

8. Conclusion

The Crop Recommendation System successfully predicted suitable crops using machine learning classification techniques. The Random Forest classifier achieved excellent accuracy and demonstrated the importance of machine learning in smart agriculture systems.

9. References

- Crop Recommendation Dataset from Kaggle

- scikit-learn documentation
- pandas documentation
- matplotlib and seaborn libraries

10. Output/Results

The screenshot shows a Google Colab notebook titled "Crop_Recommendation_System.ipynb". The code has been executed, resulting in the following output:

```
Crop_recommendation.csv
Saving Crop_recommendation.csv to Crop_recommendation (1).csv
# # # temperature humidity pH rainfall label
0 98 42 43 28.879744 82.462744 6.562885 282.935536 rice
1 85 58 43 21.72862 80.31964 7.838696 126.655337 rice
2 68 55 48 23.088459 82.128763 7.888287 263.964248 rice
3 74 75 46 26.493996 86.126363 6.986881 242.864834 rice
4 78 42 42 28.138175 81.688573 7.628473 262.717368 rice
Accuracy : 0.993318181818182
```

Classification Report

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	23
banna	1.00	1.00	1.00	23
blackgram	1.00	1.00	1.00	28
chickpeas	1.00	1.00	1.00	26
coconut	1.00	1.00	1.00	27
coffee	1.00	1.00	1.00	17
cotton	1.00	1.00	1.00	17
grapes	1.00	1.00	1.00	14
jute	0.92	1.00	0.96	23
kidneybeans	1.00	1.00	1.00	28
lentil	0.92	1.00	0.96	11
millet	1.00	1.00	1.00	22
mango	1.00	1.00	1.00	19
mushrooms	1.00	0.96	0.98	24
mustardseeds	1.00	1.00	1.00	19
muskmelon	1.00	1.00	1.00	17
orange	1.00	1.00	1.00	14
papaya	1.00	1.00	1.00	23
pigeonpeas	1.00	1.00	1.00	22
ponyrusset	1.00	1.00	1.00	23
rice	1.00	0.82	0.94	19
watermelon	1.00	1.00	1.00	19
accuracy			0.99	448
macro avg	0.99	0.99	0.99	448
weighted avg	0.99	0.99	0.99	448

Confusion Matrix

```
[ [23 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 21 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 28 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 28 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 27 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 17 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 17 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 14 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 23 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 20 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 11 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 21 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 19 0 0 0 0 0 0 0]
```